



Neuro-symbolic approaches to the SAT problem

Pattern Recognition &
Reinforcement Learning
(survey with coding project)



Donato Meoli

SAT Problem

- NeuroSAT
- Background
 - Message Passing Neural Networks (MPNNs)
- Model
- Training
- Results

Selsam, Daniel, et al., [Learning a SAT Solver from Single-Bit Supervision](#).
Gilmer, Justin, et al., [Neural Message Passing for Quantum Chemistry](#).

NeuroSAT

(ISPR - survey)

NeuroSAT

- Goal: Can a neural network learn to solve SAT problems?
- Use MPNN framework that learns to solve SAT problem after only trained as a classifier to predict satisfiability on a dataset of SAT problems.
- Train with one bit supervision for each SAT problem to indicate whether or not the problem is satisfiable.
- During test time, if the solution is *satisfiable*, the network initially guesses unsatisfiability with low confidence until it finds a solution at which point it converges and guesses satisfiability with high confidence.

Model (1)

Encode the SAT problem as an undirected graph

- One node for every clause and every literal
- Embedding for every literal and every clause
- An edge between every literal and every clause it appears in
- An edge between each pair of complementary literals

NeuroSAT iteratively refines a vector space embedding for each node by passing messages back and forth along the edges of the graph

An iteration consists of two stages:

- Each clause receives messages from its neighboring literals and updates its embedding accordingly
- Each literal receives its message from its neighboring clauses as well from its complement and update its embedding

Model (2)

The embeddings for literal and clause at each time step are represented by

$$L^{(t)} \in \Re^{2n \times d} \text{ and } C^{(t)} \in \Re^{m \times d}$$

Parametrized by three multilayer perceptrons:

$$L_{msg}, C_{msg} \text{ and } L_{vote}$$

Hidden states of these LSTM are:

Model (3)

The embeddings for literal and clause at each time step are represented by

$$L^{(t)} \in \Re^{2n \times d} \text{ and } C^{(t)} \in \Re^{m \times d}$$

Parametrized by three multilayer perceptrons:

$$L_{msg}, C_{msg} \text{ and } L_{vote}$$

Hidden states of these LSTM are:

Model (2)

A single iteration consist of applying

$$\begin{aligned}(C^{(t+1)}, C_h^{(t+1)}) &\leftarrow C_u([C_h^{(t)}, M^T \mathbf{L}_{msg}(L^{(t)})]) \\ (L^{(t+1)}, L_h^{(t+1)}) &\leftarrow L_u([L_h^{(t)}, F(L^{(t)}), M\mathbf{C}_{msg}(C^{(t+1)})])\end{aligned}$$

After T iterations, we compute:

$$\begin{aligned}L_*^{(T)} &\leftarrow \mathbf{L}_{vote}(L^{(T)}) \\ y^{(T)} &\leftarrow \text{mean}(L_*^{(T)})\end{aligned}$$

- AlphaZeroSAT
- Background
 - Monte Carlo Tree Search (MCTS)
- Model
- Training
- Results

Wang, Fei, and Tiark Rompf, [From Gameplay to Symbolic Reasoning: Learning SAT Solver Heuristics in the Style of Alpha\(Go\) Zero.](#)

AlphaZeroSAT

(ISPR + RL - survey)

AlphaZeroSAT

- Goal: Can Alpha(Go) Zero Learn a Branching Heuristic for a SAT Solver?

- GraphQSAT
- Background
 - Deep Q-Learning (DQN)
 - Graph Neural Networks (GNNs)
- Model
- Graph Attention Networks (GATs)
- Training
- Results

Kurin, Vitaly, et al., [Can Q-Learning with Graph Networks Learn a Generalizable Branching Heuristic for a SAT Solver?](#).

Battaglia, Peter W., et al., [Relational inductive biases, deep learning, and graph networks](#).

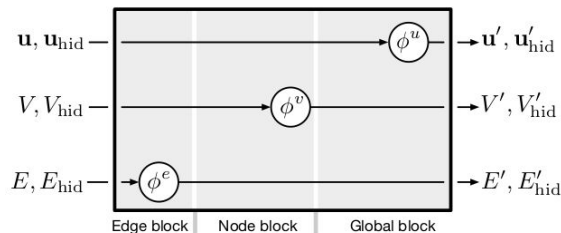
GraphQSAT

(ISPR + RL - survey with extra coding)

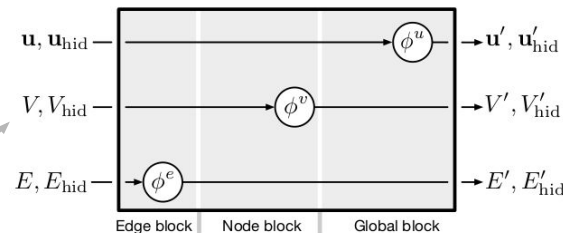
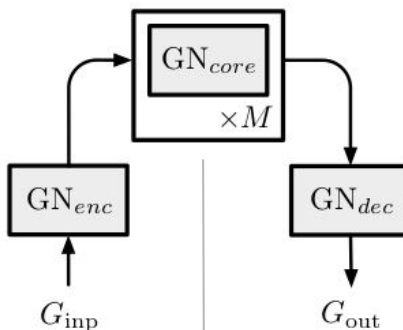
GraphQSAT

- Goal: Can Q-Learning with Graph Neural Networks Learn a Branching Heuristic for a SAT Solver?

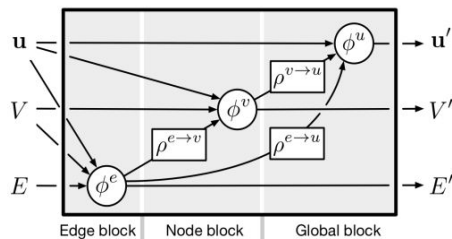
Encode-Process-Decode Architecture (GraphQSAT)



independent Graph Network



independent Graph Network



full Graph Network

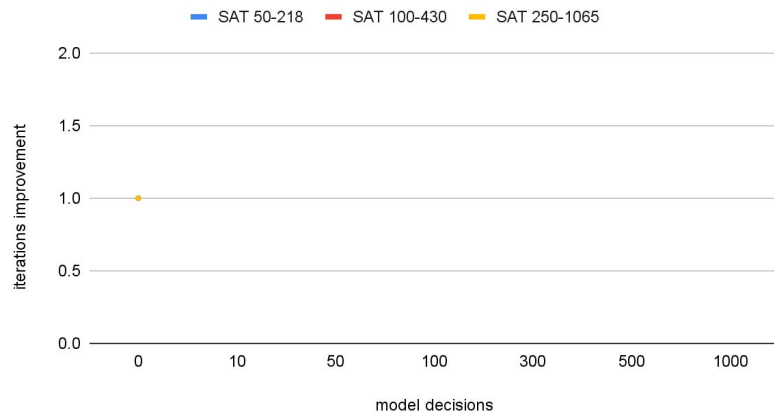
Experiments

To evaluate the GraphQSAT heuristic performance vs the VSID one, we train our agent using SAT instances from the [SATLIB benchmark problems](#).

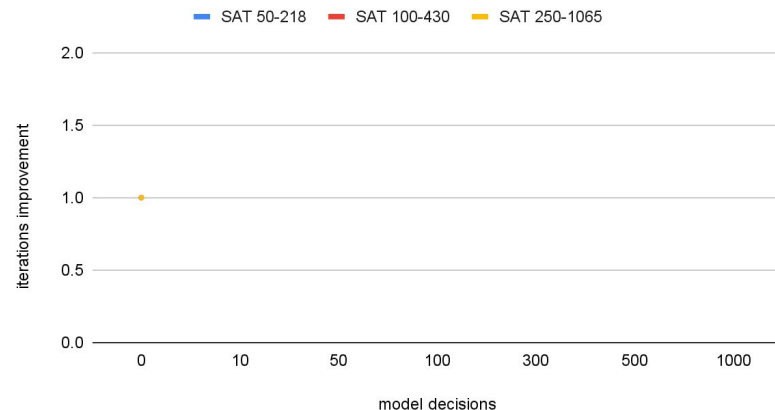
Experiments were be run on Google Colab under Nvidia Tesla P100 GPU.

GraphQSAT on *Uniform Random 3-SAT* problems

GraphQSAT trained on SAT 50-218

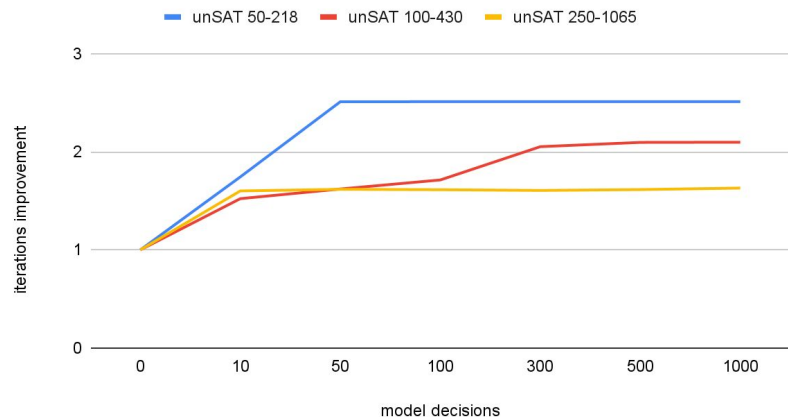


GraphQSAT trained on SAT 100-430

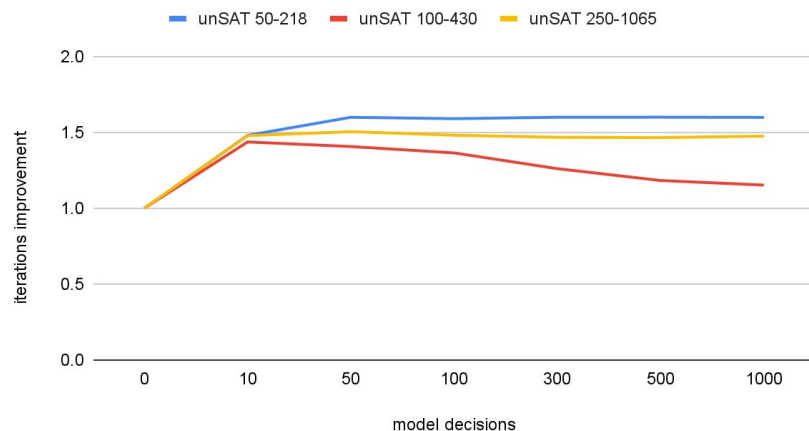


GraphQSAT on *Uniform Random 3-unSAT* problems

GraphQSAT trained on unSAT 50-218



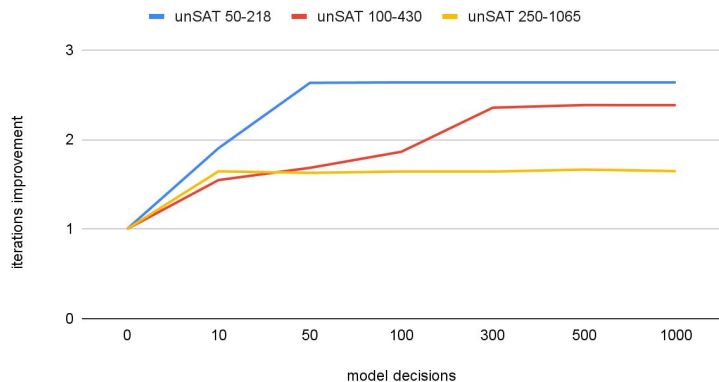
GraphQSAT trained on unSAT 100-430



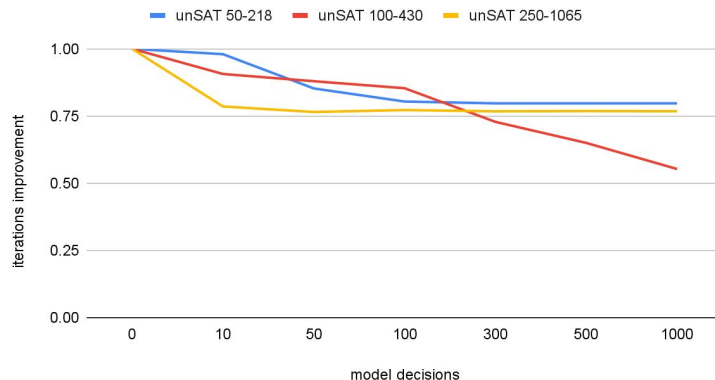
GraphQSAT generalization from SAT to unSAT

The problem formulation and representation makes it possible to solve unSAT problems when training only on SAT instances, which is problematic for some existing approaches, i.e., *NeuroSAT*.

GraphQSAT trained on SAT 50-218

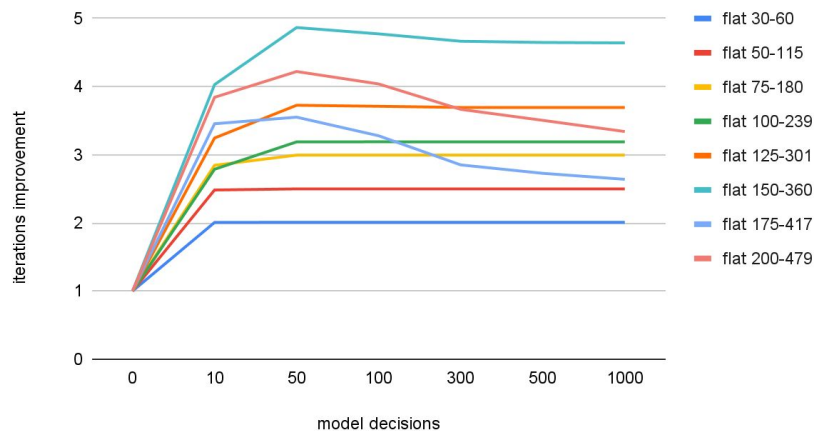


GraphQSAT trained on SAT 100-430



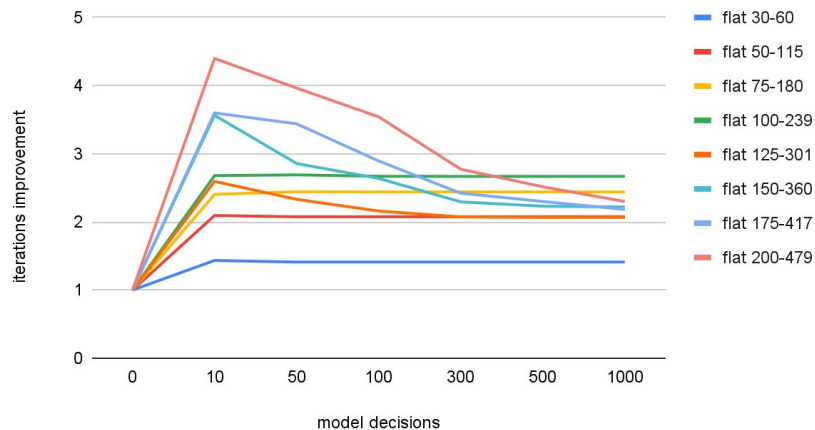
GraphQSAT on *Graph Coloring* problems

GraphQSAT trained on flat 50-115

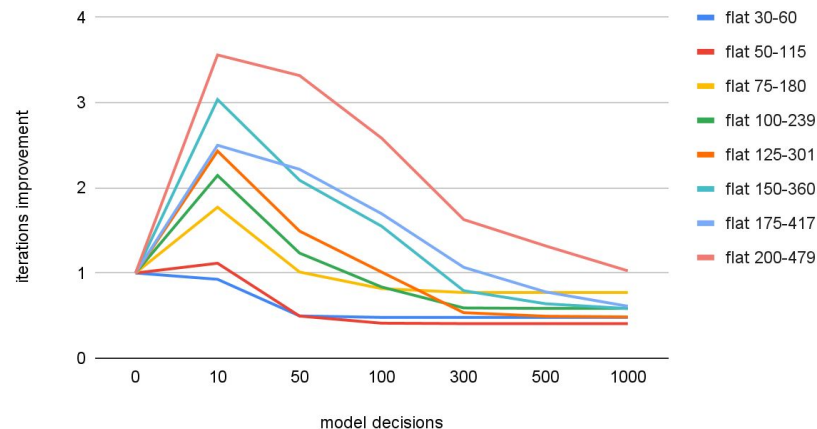


Transfer across task families of GraphQSAT

GraphQSAT trained on SAT 50-218



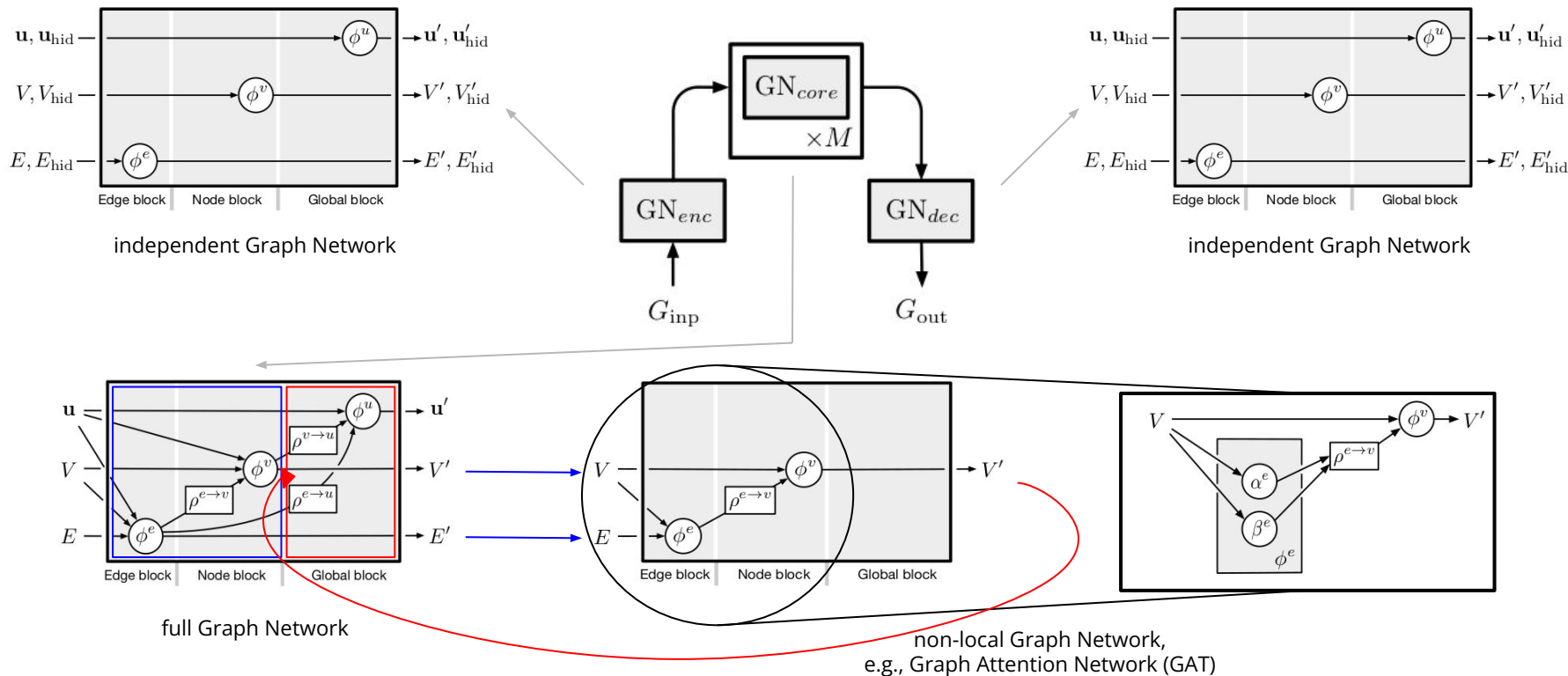
GraphQSAT trained on SAT 100-430



(Possible) Improvement idea: Graph Attention Mechanism

In order to allow the Graph Network to encode more abstract dependencies in the SAT graphs, we try to augment the Core architecture introducing a *Graph Attention Mechanism* over node and edge features.

Encode-Process-Decode Architecture (GATQSAT)



GATQSAT on *Uniform Random 3-SAT* problems

GATQSAT on *Uniform Random 3-unSAT* problems

GATQSAT generalization from SAT to unSAT

GATQSAT on *Graph Coloring* problems

Transfer across task families of GATQSAT

In summary: GraphQ₂SAT vs GATQ₂SAT

Thanks for the ATTENTION
(yours and neural)